Comment prédire la consommation électrique

Les modèles XGBoost, LightGBM et CatBoost, expliqués simplement

Projet S9 POWER · encadrants : E. Grivel, A. Leys

1 **Notre projet**
ce que nous cherchons à prédire, et pourquoi cette méthode

2 **Les bases**
l'arbre de décision, puis le boosting : des arbres qui se corrigent

3 **Trois modèles**
XGBoost, LightGBM, CatBoost : leur nom, leur intérêt, leur rôle chez nous

4 **Les comparer**
notre méthode d'expérience

Notre projet et nos données

POWER : prévoir la consommation électrique française, demi-heure par demi-heure.

Peut-on prévoir ce que la France va consommer demain ?

Pourquoi c'est utile. L'électricité ne se stocke presque pas : à chaque instant, la production doit égaler la consommation.

Indicateurs : connus à l'avance				Cible
Date et heure	Jour	Temp.	Férié	Conso (MW)
14 janv. 2025, 03:00	mardi	1 °C	non	**56 200**
14 janv. 2025, 19:00	mardi	2 °C	non	**78 100**
19 janv. 2025, 19:00	dimanche	4 °C	non	**66 300**
15 août 2025, 13:00	vendredi	28 °C	oui	**36 500**
…	…	…	…	…

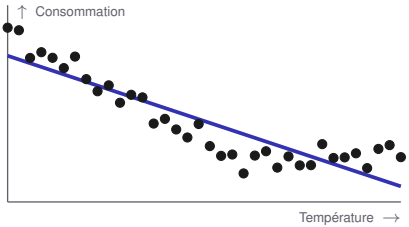
Valeurs illustratives · 1 ligne = 1 demi-heure · ≈ 250 000 lignes

1 Collecter — consommation, météo, calendrier › **2 Assembler** — une table : 1 ligne = 1 demi-heure › **3 Entraîner** — le modèle apprend sur le passé › **4 Prédire** — une année jamais vue › **5 Comparer** — à la réalité et à RTE

Pourquoi cette méthode, et pas une autre ?

La consommation ne suit pas une droite : plus il fait froid, plus on chauffe, puis la courbe devient plate.

Consommation selon la température



En bleu : ce que prédirait une simple droite. Elle se trompe presque partout.

Les autres pistes

Régression linéaire

Idée : tracer une droite entre température et consommation.

Limite : ne suit pas une courbe coudée.

Séries temporelles classiques (ARIMA)

Idée : prolonger une courbe qui se répète chaque jour et semaine.

Limite : pensées pour une seule courbe : peu adaptées à la météo.

Réseaux de neurones (LSTM)

Idée : un « cerveau artificiel » qui apprend la forme de la courbe.

Limite : gourmands en données et en réglages, peu explicables ; sur des tableaux comme le nôtre, souvent pas meilleurs que les arbres boostés.

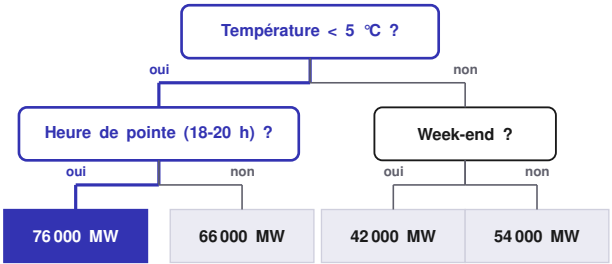
Notre choix

Des arbres de décision combinés par boosting. Ils épousent les courbes, mélangent heure, météo et calendrier, et restent explicables.

L'arbre de décision : un jeu de questions

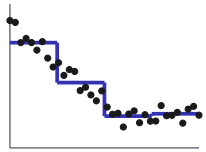
Une suite de questions oui / non. Au bout du chemin, l'arbre donne une prédiction.

Exemple : mardi, 19 h, 2 °C

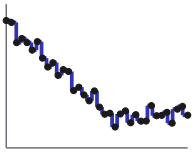


Chemin en bleu : il fait froid, c'est l'heure de pointe, donc 76 000 MW (chiffres illustratifs). On part du haut, on suit les réponses ; la feuille donne la prédiction.

Comment il apprend : il découpe



4 groupes ✓ : suit la forme



30 groupes × : par cœur

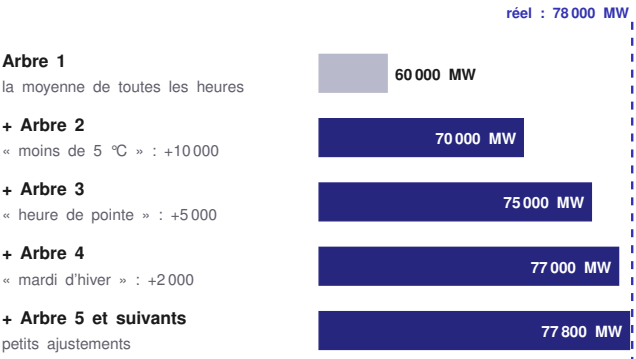
Chaque feuille contient la moyenne des consommations passées qui ressemblent à la situation.

Un seul arbre ne suffit pas

Soit il est trop simple, soit il apprend par cœur. L'idée du boosting : en combiner beaucoup de petits.

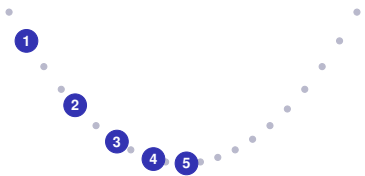
Le boosting : des arbres qui se corrigent

Exemple : un mardi à 19 h par grand froid, consommation réelle 78 000 MW (chiffres illustratifs).



Échelle à partir de 55 000 MW.

Pourquoi « gradient » ? La pente



Chaque numéro est un arbre ajouté : un pas vers le bas.

Gradient : la pente. **Learning rate** : la taille du pas. **Nombre d'arbres** : le nombre de pas.

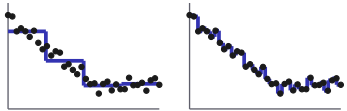
Chaque arbre corrige le reste

Un arbre seul est faible, quelques centaines bien enchaînés sont très précis. To boost, c'est renforcer.

XGBoost : le boosting « extrême »

eXtreme Gradient Boosting · créé en 2014 par Tianqi Chen · « extrême » : pousser le boosting à ses limites.

1. Il évite d'apprendre par cœur



Une pénalité sur les arbres trop compliqués : l'élève qui comprend le cours, pas celui qui récite les corrigés.

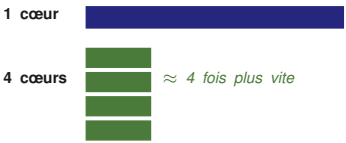
2. Il gère les valeurs manquantes



direction par défaut, apprise pour les trous

Une mesure absente (capteur en panne) ne bloque pas le modèle : chaque question a appris de quel côté envoyer les trous.

3. Il calcule en parallèle



Il construit l'arbre sur plusieurs cœurs à la fois et optimise la mémoire : bien plus rapide que le boosting d'origine.

En résumé

Le boosting classique marche, mais il est lent et apprend vite par cœur. XGBoost en est la version industrielle, plus fiable et plus rapide.

XGBoost dans notre projet

Notre modèle de référence : solide, très répandu, bien documenté.

Ce qu'on lui donne

XGBoost ne lit que des nombres : les mots (lundi, mardi…) sont d'abord transformés en nombres.

lundi	mardi	mercredi	jeudi	vendredi	samedi	dimanche
1	**2**	**3**	**4**	**5**	**6**	**7**

Exemple (valeurs illustratives)

Mardi 14 janvier 2025 · 19 h · 2 °C · non férié

XGBoost prédit 77 200 MW

réel : 78 100 MW
erreur : 1,2 %

Les 4 étapes

1. Charger la table des années passées (2012 → 2023).
2. Choisir quelques réglages : nombre d'arbres, profondeur maximale, learning rate.
3. Entraîner : les arbres s'ajoutent un à un, chacun corrige les précédents.
4. Prédire 2024, mesurer l'erreur (MAPE : erreur moyenne en %) et la comparer à RTE.

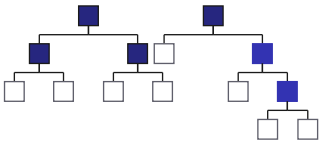
Son rôle chez nous

La référence, avec les mêmes données et les mêmes découpages que les deux autres modèles.

LightGBM : le boosting « léger »

Light Gradient Boosting Machine · « light » = léger : rapide et économe en mémoire · Microsoft, 2016.

1. Il coupe feuille par feuille



par niveau **par feuille (LightGBM)**

Il ne coupe que la feuille qui réduit le plus l'erreur : l'erreur baisse plus vite. Attention au par-cœur : on limite `num_leaves`.

2. Il range les valeurs en tiroirs

Sans tiroirs : on teste chaque valeur



Avec tiroirs (environ 255) : on teste les frontières



La température est rangée en environ 255 tiroirs : on teste seulement les frontières. Bien plus rapide, bien moins de mémoire.

3. Il se concentre sur ses erreurs

mal prédites : toutes gardées **déjà bien prédites : un échantillon**



● gardé ○ ignoré (tiré au hasard)

GOSS : on garde toutes les lignes mal prédites et un échantillon des autres. L'entraînement se concentre sur ce qui reste à apprendre.

Performance

Le benchmark un entraînement plus de 100 fois plus rapide que XGBoost,

LightGBM dans notre projet

Notre modèle pour tester beaucoup d'idées vite, et pour estimer l'incertitude.

1. Tester vite : indicateurs un à un

Base : heure, jour, mois

\+ température

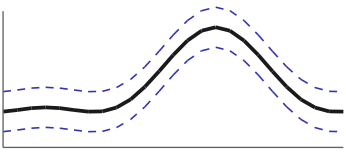
\+ jours fériés

\+ vacances

à chaque ligne : ré-entraîner, mesurer l'erreur

Un modèle rapide permet de refaire ce test des dizaines de fois : on garde un indicateur seulement s'il fait baisser l'erreur sur 2024.

2. Une fourchette de valeurs



ligne : prévision centrale · pointillés : fourchette

Avec trois modèles (quantiles 10, 50 et 90 %) : une prédiction basse, centrale et haute, par exemple « entre 70 000 et 80 000 MW ».

3. Savoir où il peut se tromper

! Trop de feuilles : il apprend par cœur

! Hors des valeurs vues : il sous-estime un record de froid

! Importance des variables : lire le gain, pas le nombre de coupures

Parades : arrêt anticipé sur l'année de validation ; importance mesurée par le gain ou par SHAP.

Son rôle chez nous

Le modèle d'exploration (ablation rapide) et des bandes d'incertitude pour le tableau de bord.

CatBoost : le boosting des catégories

Categorical Boosting · « cat » vient de catégorie (pas de chat) · créé par Yandex en 2017.

1. Il lit directement les catégories

Sans : 7 colonnes 0 / 1

1	0	0	0	0	0	0
0	**1**	0	0	0	0	0
0	0	**1**	0	0	0	0

lun · mar · mer…

Avec : des mots

lundi
mardi
…

Jour, mois, zone de vacances sont des catégories : CatBoost les gère tout seul, sans les transformer en nombres.

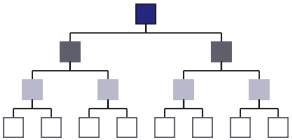
2. Il évite de tricher

1	2	3	4	5	6
utilisées				**à prédire**	pas vue

Pour la ligne 5, il n'utilise que les lignes 1 à 4.

Ailleurs, une ligne peut « se voir » pendant l'entraînement et gonfler le score. CatBoost la corrige avec un modèle qui ne l'a pas vue.

3. Des arbres symétriques



même question sur tout un étage

À chaque étage, la même question partout : arbre régulier, prédiction très rapide, moins de risque d'apprendre par cœur.

En général

De bons résultats avec peu de réglages, mais un entraînement souvent plus lent que celui de LightGBM.

CatBoost dans notre projet

Notre modèle pour les variables qui sont des mots, avec peu de réglages.

Nos colonnes : nombres ou catégories ?

heure	nombre	19
jour de la semaine	catégorie	mardi
mois	catégorie	janvier
férié	catégorie	oui / non
zone de vacances	catégorie	A, B ou C
température	nombre	2 °C
consommation 7 jours avant	nombre	74 500 MW

Comment on l'utilise

On indique les colonnes catégorielles :

```
cat_features = [jour, mois, férié, zone]
```

- Pas besoin de transformer les mots en nombres.
- Premier résultat souvent bon sans long réglage : pratique pour comparer.
- À surveiller : un entraînement plus lent, à prévoir dans nos tests.

Son rôle chez nous

Vérifier si traiter jour, mois et vacances comme de vraies catégories améliore l'erreur par rapport à XGBoost et LightGBM.

Comparaison et méthode

	XGBoost	LightGBM	CatBoost
Son nom	eXtreme Gradient Boosting	Light Gradient Boosting Machine	Categorical Boosting
Créé	2014 · Tianqi Chen	2016 · Microsoft	2017 · Yandex
Spécialité	Fiabilité : pénalité contre le par-cœur, gère les trous	Vitesse et mémoire : feuille par feuille, tiroirs, GOSS	Catégories lues directement, ordered boosting
Arbres	Niveau par niveau	Feuille par feuille	Symétriques
À surveiller	Mots à transformer en nombres	Par-cœur si trop de feuilles	Entraînement plus lent
Chez nous	**Modèle de référence**	**Exploration rapide et fourchettes**	**Jour, mois et vacances en catégories**

Même parcours pour les trois : on ajoute un indicateur à la fois, on garde celui qui fait baisser l'erreur sur 2024, puis on teste une seule fois sur 2025.

Base : heure, jour, mois › + température › + jours fériés › + vacances › + valeurs passées



Le modèle apprend sur le passé ; on règle sur 2024 ; verdict final sur 2025, une seule fois.